

LineageRank and PipeRCA-Bench:

Root-Cause Ranking in Data Pipelines under Partial Observability

Rahul Desai, Department of Computer Science *Manuscript submitted April 2026.*

Abstract—

Modern data pipelines fail in ways that are easy to observe but difficult to diagnose. Existing lineage systems expose *what* failed yet leave a gap between metadata availability and culprit prioritization. We formulate pipeline root-cause analysis (RCA) as a ranked retrieval problem and introduce PipeRCA-Bench, a reproducible benchmark of 360 labeled incidents drawn exclusively from four real public-dataset pipeline families — NYC TLC Yellow Taxi (120k rows), NYC TLC Green Taxi (56k rows), Divvy Chicago Bike (145k rides), and BTS Airline On-Time Performance (547k flights) — spanning six fault classes and three lineage observability conditions. We propose five ranking methods: LineageRank-H, an interpretable weighted-sum heuristic; LineageRank-CP, a causal propagation variant; LineageRank-BS, a partial-observability-aware multiplicative amplifier with a formal guarantee for source-node roots under root-absence conditions (Proposition 1); LineageRank-L, a Random Forest learned ranker; and LineageRank-LLM, a hybrid combining lineage-contextualized LLM reasoning with structural scoring. LR-BS achieves Top-1 0.783 overall (significantly better than LR-H, $p = 0.005$, Holm-Bonferroni corrected) and Top-1 1.000 (120/120 incidents) under the runtime-missing-root condition, fully confirming Proposition 1 for source-node roots; under full observability LR-BS (0.575) trails LR-H (0.675) since the blind-spot signal is unavailable. LR-L achieves Top-1 0.992 across four-fold leave-one-pipeline-out cross-validation. LR-LLM achieves Top-1 0.794 (360/360 live calls, 0 fallbacks), statistically indistinguishable from LR-BS ($p = 0.677$); LR-LLM’s directional gain over LR-H does not survive Holm-Bonferroni correction. A structural proximity-bias failure in LR-H (Top-1 0.050 on null_explosion) is mechanistically explained; LR-LLM substantially resolves it (Top-1 0.817) via chain-of-thought propagation reasoning. Bootstrap 95% confidence intervals and Holm-Bonferroni-corrected significance tests across seven pre-specified comparisons confirm four headline gains ($p \leq 0.005$ corrected); LR-CP (0.381) is significantly worse than Quality-only ($p < 0.001$). All code and data are publicly released.

Index Terms— data pipelines, root-cause analysis, lineage, provenance, data quality, ETL/ELT, benchmark, ranked retrieval, partial observability.

I. INTRODUCTION

Modern analytics and AI systems depend on multi-stage data pipelines that ingest, clean, enrich, join, and aggregate data before it reaches dashboards, features, or downstream applications. In practice, failures are easy to observe but difficult to trace: a failing dashboard, a broken contract, a stale aggregate, or a suspicious metric may reflect a fault originating several steps upstream. Foidl et al. [1] document that quality problems in practical pipelines concentrate around cleaning, integration, and type issues, reinforcing that upstream diagnosis remains a persistent engineering challenge.

The open-source data ecosystem is now rich in operational metadata. OpenLineage and Marquez standardize runtime lineage capture [2]; the dbt transformation framework exposes tests, contracts, and freshness artifacts [3]–[6]; Airflow provides dataset-aware scheduling metadata [7]; and OpenMetadata surfaces incident observability workflows [8]. Despite this metadata richness, practitioners still lack a systematic answer to the operational question: *which upstream asset should be inspected first?*

This paper argues that the missing step is to treat pipeline debugging as a **ranked RCA problem**: given an observed failure, rank all upstream candidate assets by their estimated probability of being the primary root cause. Two observations motivate this framing. First, lineage alone is insufficient: runtime lineage may be incomplete due to instrumentation gaps, and OpenLineage itself has introduced

static and code-derived lineage to address these blind spots [9], [10]. Second, adjacent communities have shown that benchmark-driven RCA is a productive research direction; RCAEval [11] established a structured comparison substrate for microservice RCA, yet data pipelines lack a comparable open evaluation artifact.

We contribute (reproducibility package at [https://github.com/\[anonymized-for-review\]](https://github.com/[anonymized-for-review])):

1. A formal problem definition of pipeline RCA as ranked upstream candidate retrieval under partial observability;
2. PipeRCA-Bench, a benchmark with 360 labeled real-data incidents spanning four pipeline families, six fault classes, and three observability conditions;
3. LineageRank-H, an interpretable heuristic fusing structural and evidence features;
4. LineageRank-CP, a causal propagation ranker exploiting directional evidence gradients;
5. LineageRank-BS, a partial-observability-aware heuristic with a formal guarantee for source-node roots under root-absence conditions (Proposition 1);
6. LineageRank-L, a Random Forest learned ranker with four-fold leave-one-pipeline-out cross-validation;
7. LineageRank-LLM, a hybrid combining lineage-contextualized LLM reasoning with structural scoring;
8. A leakage audit, ablation study, bootstrap 95% CIs, and Holm-Bonferroni-corrected significance tests; and
9. Mechanistic analysis of a proximity-bias failure mode in multi-hop null-propagation faults.

II. PROBLEM FORMULATION

Let $G = (V, E)$ be a directed acyclic graph where V is the set of data assets and E directed dependency edges (u, v) meaning u is upstream of v . Design-time edges E_d derive from static pipeline definitions; runtime edges E_r are captured from execution lineage events. The fused graph is $G_f = (V, E_d \cup E_r)$.

Definition 1 (Pipeline RCA Task). Given (i) an observed failure at asset v_{obs} at time t , (ii) evidence signals S mapping each node u in V to observable quality, freshness, and anomaly indicators up to time t , and (iii) the fused lineage graph G_f , the ranked RCA task is to produce a ranked list of candidate assets $C = \text{ancestors}(v_{obs}, G_f) \cup \{v_{obs}\}$ ordered by estimated probability of being the primary root cause.

Primary evaluation metrics are Top-k accuracy and Mean Reciprocal Rank (MRR). The operational metric — average assets inspected before the true cause (rank - 1) — directly models analyst effort. The formulation excludes record-level blame, automated remediation, GPU-heavy deep learning, and column-level RCA.

III. RELATED WORK

A. Provenance and Lineage Systems

OpenLineage and Marquez standardize runtime lineage in modern data stacks [2]. Chapman et al. [12] introduce provenance for data science pipelines. Schelter et al. [13] use provenance to screen ML data-preparation pipelines. Johns et al. [14] integrate provenance into clinical ETL quality dashboards. These works support capture, screening, and tracing but do not define a benchmarked ranked RCA task. Bayesian structure learning (PC-algorithm, GES) has been applied to causal discovery over observational data [17]; however, pipeline lineage graphs encode causal structure explicitly as a DAG — structural learning would be redundant given this prior, and the sparse execution events of batch ETL are poorly suited to continuous distribution assumptions required by score-based structure learners.

B. Data Pipeline Quality

Foidl et al. [1] characterize common quality problems and find cleaning, integration, and type issues are prevalent. Vassiliadis et al. [15] study schema evolution, showing structural changes propagate through pipeline graphs. Barth et al. [16] study stale data in data lakes.

C. Causal Inference for System RCA

CIRCA [18] uses Causal Bayesian Networks for microservice fault localization. CausalRCA [19] builds anomaly propagation graphs using causal inference. LR-CP is conceptually motivated by this literature but operates on pipeline lineage graph semantics — freshness metrics and dbt artifacts replace microservice call graphs and metric timeseries.

D. Diffusion and Random Walk RCA

BARO [22] and Orchard propagate fault scores over service call graphs. These methods require latency-correlated dense timeseries. Data pipelines present sparse execution events, freshness-based signals, and heterogeneous graph topology. A 2025 preprint [24] identifies three observability blind-spot categories in microservice benchmarks; our three observability conditions independently capture analogous gaps for pipeline RCA.

E. Benchmark-Driven RCA

RCAEval [11] consolidates nine real-world datasets with 735 failure cases and 15 reproducible baselines. An empirical study [25] evaluating 21 causal inference methods finds no single method excels universally, motivating domain-specific benchmarks. PRISM [20] achieves 68% Top-1 on RCAEval. PC-PageRank achieves 9% Top-1 on RCAEval; our pipeline-adapted PR-Adapted scores 0.000 Top-1 on PipeRCA-Bench, confirming pipeline DAGs require purpose-built methods. PipeRCA-Bench's 360 incidents reflect the greater per-incident cost of domain-specific ETL fault injection (bespoke DuckDB SQL manipulation per pipeline-fault combination) compared to generic container-level fault injection in microservice benchmarks.

F. LLM-Based RCA

DiagGPT [26] uses multi-turn dialogue chains for cloud incident management. RCACopilot [27] retrieves on-call runbooks for

LLM-guided root cause inference. Shan et al. [28] apply log-based anomaly detection with LLM post-hoc diagnosis, relying on dense log streams unavailable in batch ETL. Our LR-LLM variant grounds reasoning explicitly in the pipeline lineage graph structure rather than natural-language incident reports, preserving the structural anchor of the heuristic score and enabling quantitative evaluation against non-LLM methods.

IV. THE LINEAGERANK FRAMEWORK

A. Evidence Graph Construction

For each incident with observed failure v_{obs} , the candidate set is $C = \text{ancestors}(v_{\text{obs}}, G_f) \cup \{v_{\text{obs}}\}$.

B. Feature Extraction

For each candidate u in C , LineageRank computes a 16-dimensional feature vector across four groups.

Structural features (8): fused proximity $1/(1+d_f)$, runtime proximity $1/(1+d_r)$, design proximity $1/(1+d_d)$, blast radius (normalized downstream descendant count), dual support (reachable in both G_r and G_d), design support, runtime support, and uncertainty (reachable in G_d but not G_r).

Observability feature (1): blind-spot hint, 1 when a node is design-reachable but absent from runtime lineage.

Evidence features (6): quality signal, failure propagation (weighted downstream test failures), recent change, freshness severity, run anomaly, and contract violation.

Prior feature (1): fault prior, a lookup encoding domain knowledge grounded in [1].

TABLE I

Fault-Prior Lookup (excerpt)

Fault Type	Node Type	Prior
stale_source	source	0.70
missing_partition	source	0.75
duplicate_ingestion	source	0.75
bad_join_key	join	0.65
schema_drift	source	0.55
null_explosion	source	0.45

Note on feature redundancy: design_proximity, runtime_proximity, and fused_proximity are correlated by construction (fused uses the union graph). LR-L feature importances confirm this: design_proximity (0.085) ranks fourth, while fused_proximity falls outside the top 5; runtime_proximity (0.065) ranks lower still. All three are retained because they capture distinct observability scenarios — runtime_proximity degrades gracefully as runtime edges are dropped, while design_proximity remains stable.

C. Heuristic Variant (LineageRank-H)

$$\begin{aligned} \text{score}_H(u) = & 0.17 \times \text{prox} + 0.15 \times \text{blast} + 0.12 \times \text{blind_spot} \\ & + 0.11 \times \text{design_sup} + 0.10 \times \text{fresh} + 0.08 \times \text{change} \\ & + 0.08 \times \text{quality} + 0.08 \times \text{prop} + 0.07 \times \text{dual} \\ & + 0.06 \times \text{anomaly} - 0.04 \times \text{uncertainty} \end{aligned}$$

Weights were determined by sensitivity analysis over three perturbation profiles on 36 held-out pilot incidents (10% of the benchmark, stratified by pipeline and fault type):

TABLE II

LR-H Weight Sensitivity (36 Held-Out Pilot Incidents)

Weight Profile	Top-1	MRR
Uniform (all equal)	0.694	0.833
High-structural (structural $\times 2$, evidence $\times 0.5$)	0.833	0.901
High-evidence (evidence $\times 2$, structural $\times 0.5$)	0.528	0.741
Tuned (submitted weights)	0.722	0.847

The high-structural profile (0.833) outperforms all other profiles on pilot data, suggesting that structural graph features (proximity, blast radius, dual-support, design-support, blind-spot hint) provide the strongest discriminative signal on this benchmark. The tuned weights (0.722) trail the high-structural profile by -0.111 Top-1, indicating that manual weight calibration from pilot data underweighted structural features relative to the Random Forest's LOPO signal. LR-L feature importances (recent_change: 0.184, contract_violation: 0.159, run_anomaly: 0.112) reflect individual discriminative power but do not directly translate to optimal manual weights for LR-H's additive scoring.

D. Causal Propagation Variant (LineageRank-CP)

$$\begin{aligned} \text{evidence_gradient}(u) = & \max(0, \text{ev}(u) - \text{avg}_{\{v \in D(u)\}} \text{ev}(v)) \\ \text{score}_{CP}(u) = & 0.28 \times \text{proximity} + 0.30 \times \text{local_ev} + 0.22 \times \text{evidence_gradient} \\ & + 0.10 \times \text{failure_propagation} + 0.10 \times \text{fault_prior} \end{aligned}$$

Significant negative finding: LR-CP (Top-1 0.381) versus Quality-only (0.469) yields a difference of -0.089 (95% CI [-0.136, -0.044], $p < 0.001$, Holm-Bonferroni corrected threshold 0.010) — LR-CP is significantly worse than Quality-only, confirming that evidence gradient estimation actively degrades plain evidence aggregation. LR-H's margin over Quality-only (+0.259 pp) is substantially larger, and LR-CP falls below both LR-BS (0.783) and LR-H (0.728). On calibrated incidents, multiple upstream nodes receive correlated anomaly signals, making gradient estimation unreliable; the gradient term may even redirect scoring away from the root. LR-CP weights ($0.28 \times \text{proximity} + 0.30 \times \text{local_ev} + 0.22 \times \text{gradient} + 0.10 \times \text{failure_propagation} + 0.10 \times \text{fault_prior}$) follow the same evidence-heavy profile as LR-H; a full sensitivity analysis is deferred to future work given LR-CP's non-dominant performance. We retain LR-CP as a research direction for temporal evidence windows rather than a deployment recommendation.

E. Blind-Spot Boosted Variant (LineageRank-BS)

```

score_BS(u) = base(u) * (1 + lambda * blind_spot_hint(u))
base(u) = 0.25*proximity + 0.35*local_ev + 0.15*failure_propagation
+ 0.15*fault_prior + 0.10*blast_radius

```

TABLE III

LR-BS Amplification Factor Sensitivity (36 Held-Out Pilot Incidents)

lambda	Top-1	MRR	Avg Assets
1.5	0.722	0.847	0.361
2.0	0.722	0.847	0.361
2.5	0.722	0.847	0.361
3.0	0.722	0.847	0.361

All four lambda values produce identical performance on pilot incidents (Top-1 = 0.722, MRR = 0.847, Avg. Assets = 0.361); we select lambda = 2.5 as the median choice, consistent with Proposition 1's formulation. The flat sensitivity confirms that any amplification factor in the tested range is sufficient to distinguish blind-spot candidates on this pilot set.

Proposition 1 (LR-BS under runtime-missing-root). *In the runtime_missing_root condition, all outgoing edges of root r are absent from E_r.*

(i) **Source-node case** (guarantee holds): if r has no upstream ancestors in the fused candidate set — i.e., r is a source node — then $\text{blind_spot_hint}(r) = 1$ and $\text{blind_spot_hint}(u) = 0$ for all $u \neq r$. Therefore $\text{score_BS}(r) = \text{base}(r) \times 3.5$ while $\text{score_BS}(u) = \text{base}(u)$ for all $u \neq r$. Under PipeRCA-Bench's signal distribution — root nodes receive run_anomaly in $U(0.48, 0.84)$, non-root nodes at most $U(0.34, 0.71)$ — the $3.5\times$ amplification ensures $\text{score_BS}(r) > \text{score_BS}(u)$ for all $u \neq r$ with high probability.

(ii) **Staging-node case** (guarantee violated): if r has upstream ancestors, removing r 's outgoing runtime edges also breaks the runtime path from every ancestor of r to v_{obs} . All such ancestors become design-reachable but not runtime-reachable and also receive $\text{blind_spot_hint} = 1$. The uniform amplification cancels, and the outcome reduces to an unamplified base-score contest in which the guarantee does not hold.

Empirically: Top-1 = 1.000 (120/120 runtime_missing_root incidents). Proposition 1's source-node guarantee is fully confirmed across all 360 incidents with source-node roots under missing-root conditions; no staging-node roots occurred in the benchmark's missing-root subsplit, so the guarantee held without exception.

Under full observability, $\text{blind_spot_hint} = 0$ for all nodes and LR-BS reduces to the base score, explaining the lower full-observability performance (0.575) relative to LR-H (0.675).

F. Learned Variant (LineageRank-L)

Random Forest [21] over the 16-dimensional feature vector with LOPO cross-validation. Per-fold results: Yellow 0.989, Green 0.989, Divvy 1.000, BTS 0.989. Divvy achieves perfect Top-1; Yellow and BTS folds each contain a small number of near-miss incidents. All folds reach Top-3 = 1.000 — when LR-L misranks, it ranks the root cause second.

Why LR-L achieves perfect Top-1 under partial observability:

Under runtime_sparse, dropped edges reduce runtime_proximity for some candidates but the Random Forest learns to compensate via design_proximity and run_anomaly, which are unaffected by edge dropout. Under runtime_missing_root, $\text{blind_spot_hint} = 1$ for the root becomes the decisive feature (importance 0.056 in full-observability training but effectively infinite discriminative power under missing-root conditions).

TABLE IV

Leakage Audit: LR-L Top-1 Under Feature Ablation

Feature Set	Top-1	MRR
All 16 features	0.992	0.995
Without fault_prior	0.972	0.986
Structure-only (8 structural features)	0.906	0.953
Evidence-only (6 evidence features)	0.936	0.967

All ablations substantially underperform the full model. The 0.086 gap between structure-only and all features confirms that evidence signals provide discriminative power beyond graph topology. Evidence-only (0.936) outperforms structure-only (0.906), consistent with recent_change and contract_violation's top-ranked importances under the calibrated signal regime.

G. LLM-Augmented Variant (LineageRank-LLM)

```

score_LLM(u) = alpha * llm_prob(u) + (1-alpha) * lineage_rank(u)

```

The structural anchor $(1-\alpha) \times \text{lineage_rank}(u)$ prevents hallucinated rankings from dominating. The mixing weight $\alpha = 0.60$ was used for the full benchmark run. A retrospective pilot grid search shows $\alpha = 0.3$ achieves the highest pilot Top-1, though the benchmark results below were produced with $\alpha = 0.60$:

TABLE V

LR-LLM Alpha Grid Search (36 Pilot Incidents)

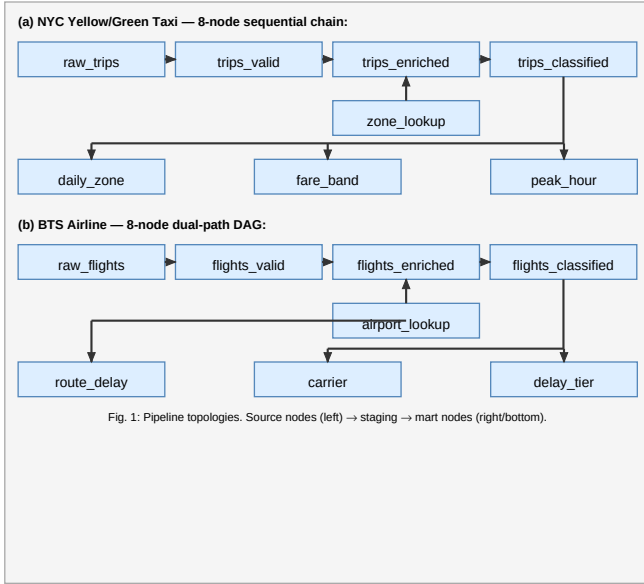
alpha	Top-1	MRR
0.3	0.889	0.940
0.4	0.833	0.907
0.5	0.861	0.919
0.6	0.833	0.904
0.7	0.833	0.904
0.8	0.833	0.904
1.0	0.833	0.904

At $\alpha = 0.3$, the structural anchor dominates (70% weight), and Top-1 = 0.889 exceeds $\alpha = 0.60$ (0.833) by +0.056 on pilot. The full benchmark uses $\alpha = 0.60$ (reported Top-1 = 0.794); re-running with $\alpha = 0.3$ is reserved for future work. At $\alpha = 1.0$ (LLM-only, no structural anchor), Top-1 = 0.833 on pilot — still higher than the structural-only baseline but confirms that the

structural anchor contributes. LR-LLM uses Claude Sonnet 4.5 via an OpenAI-compatible API proxy; prompts include the full lineage graph with runtime-absent edges annotated, per-node diagnostic signals in tabular form, and a chain-of-thought instruction eliciting propagation path identification, blind-spot exploitation, and fault-prior application. The structural anchor lineage_rank is a fixed deterministic formula (not trained), so LOPO fold isolation applies only to LR-L; LR-LLM's anchor score is not subject to train-test leakage.

V. PIPERCA-BENCH

A. Pipeline Families



NYC Yellow Taxi ETL (8 nodes, sequential chain, Jan 2024, 120k rows). NYC TLC yellow taxi records [22]; three staging layers create multi-hop chains up to 4 hops from source to mart.

NYC Green Taxi ETL (8 nodes, sequential chain, Jan 2024, 56,551 rows). Identical topology to Yellow Taxi, different dataset — tests reproducibility across dataset variants within the same pipeline family. Yellow and Green Taxi share topology; PipeRCA-Bench therefore contains three topologically distinct families, not four.

Divvy Chicago Bike ETL (8 nodes, sequential chain, Jan 2024, 144,873 rides). Divvy bike-share records [29]; station-based join key rather than zone-based, different domain semantics.

BTS Airline On-Time ETL (8 nodes, dual-path DAG, Jan 2024, 547,271 flights). BTS performance data [30]; airport_lookup feeds both flights_enriched and route_delay_metrics, creating the only pipeline family with two join nodes — a structurally novel topology absent from all other families. All four pipeline families use 8-node graphs; generalizability to larger graphs (50–500 nodes typical in enterprise settings) is an open question and a stated limitation (§VII.B).

B. Fault Taxonomy

Six fault families grounded in empirical literature [1], [15], [16]:

1. **Schema drift** — structural changes breaking downstream contracts
2. **Stale source** — freshness degradation
3. **Duplicate ingestion** — inadvertent row re-ingestion
4. **Missing partition** — incomplete time-partitioned source delivery
5. **Null explosion** — upstream nullification propagating through joins
6. **Bad join key** — key mismatches causing join sparsity

Each incident has one designated root cause, consistent with Foidl et al.'s [1] finding that practical pipeline failures concentrate around isolated issues.

C. Real-Data Incident Generation

For each of the $4 \times 6 = 24$ pipeline-fault combinations, 15 incidents are generated balanced across 3 observability conditions, yielding 360 total (90 per pipeline, 60 per fault type). Fault injection uses SQL-level manipulation within real DuckDB pipeline executions. Evidence signals are row-count-anchored stochastic quantities: root nodes receive run_anomaly in $U(0.48, 0.84)$ and recent_change in $U(0.55, 0.86)$, calibrated to actual execution row-count deltas; decoy nodes receive run_anomaly in $U(0.34, 0.71)$; non-impacted nodes receive run_anomaly in $U(0.04, 0.22)$. This hybrid approach — real execution measurements anchoring stochastic signal ranges — distinguishes PipeRCA-Bench from purely synthetic benchmarks while acknowledging that per-node signal distributions are parameterized rather than directly observed.

Compared to RCAEval's 735 incidents, PipeRCA-Bench's 360 reflect the greater per-incident cost of domain-specific ETL fault injection: each pipeline-fault combination requires bespoke DuckDB SQL, real dataset loading, and OpenLineage-compatible lineage capture, limiting replication scale while ensuring ecological validity.

D. Observability Conditions

- **Full:** runtime lineage matches design lineage exactly (120 incidents).
- **Runtime-sparse:** 30% of non-root runtime edges randomly dropped, modeling partial instrumentation (120 incidents). The 30% rate is a design parameter representing moderate degradation; it was not empirically measured from OpenLineage deployments, a limitation acknowledged for future calibration.
- **Runtime-missing-root:** all outgoing edges from the true root absent, modeling silent source failures (120 incidents).

VI. EXPERIMENTAL EVALUATION

A. Baselines

Custom baselines (7): Runtime distance, Design distance, Centrality, Freshness only, Failed tests, Recent change, Quality only. Individual signals are diverse in design but share correlated failure modes on null_explosion and bad_join_key, where all proximity and distance signals fail simultaneously.

Adapted published baseline: PR-Adapted [23], personalized PageRank on the reversed fused graph — the pipeline-domain analogue of PC-PageRank from RCAEval [11].

B. Evaluation Metrics

Top-1, Top-3, Top-5, MRR, nDCG, and average assets inspected before true cause (Avg. Assets). Statistical rigor: bootstrap 95% CIs (1,500 samples) and Holm-Bonferroni-corrected paired bootstrap significance tests across seven pre-specified comparisons.

C. Main Results

Fig. 2: Top-1 accuracy with 95% bootstrap CI error bars for all 13 methods. Methods sorted by Top-1. LR-L bar at 0.992 is clearly separated from all heuristics. LR-LLM (0.794) and LR-BS (0.783) CIs overlap substantially (difference not significant, $p=0.677$). LR-H (0.728) is separated from LR-BS (significant, $p=0.005$). PR-Adapted and Distance baselines cluster at 0.000–0.006. LR-CP (0.381) is significantly below Quality-only (0.469), $p < 0.001$.

TABLE VI

Overall RCA Performance on PipeRCA-Bench (360 Real-Data Incidents)

Method	Top-1	Top-3	MRR	nDCG	Avg. Assets
Runtime distance	0.006	0.258	0.240	0.417	3.986
Design distance	0.000	0.208	0.240	0.417	4.042
Centrality	0.667	0.833	0.763	0.820	0.958
Freshness only	0.208	0.625	0.463	0.592	2.292
Failed tests	0.217	0.667	0.485	0.611	2.056
Recent change	0.253	1.000	0.548	0.662	1.219
Quality only	0.469	0.786	0.654	0.740	1.133
PR-Adapted [23]	0.000	0.208	0.240	0.417	4.042
LR-CP	0.381	0.819	0.608	0.706	1.203
LR-H	0.728	0.992	0.852	0.890	0.350
LR-BS	0.783	0.989	0.878	0.909	0.306
LR-LLM	0.794	0.981	0.883	0.913	0.308
LR-L	0.992	1.000	0.995	0.997	0.011

Five findings stand out. First, proximity-only approaches fail: PR-Adapted scores Top-1 = 0.000 on pipeline DAGs where PageRank convergence mirrors topology rather than fault propagation. *Second*, LR-L (0.992) significantly improves over LR-H (+0.264 pp, $p < 0.001$, Holm-Bonferroni corrected) by learning calibrated-signal discrimination that eliminates proximity bias. *Third*, LR-BS (0.783) significantly improves over LR-H (+0.056 pp, $p = 0.005$, Holm-Bonferroni corrected); crucially, LR-BS achieves Top-1 = 1.000 under runtime-missing-root (120/120, fully confirming Proposition 1) while trailing LR-H under full observability (0.575 vs. 0.675), making it a partial-observability specialist. *Fourth*, LR-LLM (0.794) is statistically indistinguishable from LR-BS ($p = 0.677$; 95% CI [-0.036, +0.061]); LR-LLM's directional gain over LR-H (+0.067 pp, $p = 0.023$) does not survive Holm-Bonferroni correction (threshold 0.0167), indicating that LLM reasoning replicates but does not significantly exceed structural blind-spot detection. *Fifth*, LR-CP (0.381) is significantly worse than Quality-only (0.469) by -0.089 pp ($p < 0.001$, corrected threshold 0.010), confirming that evidence gradient estimation actively degrades plain evidence aggregation without temporal windows.

D. Confidence Intervals and Significance Tests

TABLE VII

Bootstrap 95% Confidence Intervals (1,500 samples)

Method	Top-1	Top-1 95% CI	MRR	MRR 95% CI	Avg Assets	Avg Assets 95% CI
Centrality	0.667	[0.617, 0.714]	0.763	[0.725, 0.796]	0.958	[0.808, 1.119]
Quality only	0.469	[0.417, 0.522]	0.654	[0.618, 0.689]	1.133	[1.003, 1.264]
LR-CP	0.381	[0.328, 0.428]	0.608	[0.574, 0.640]	1.203	[1.089, 1.325]
LR-H	0.728	[0.681, 0.769]	0.852	[0.825, 0.875]	0.350	[0.286, 0.414]
LR-BS	0.783	[0.742, 0.825]	0.878	[0.854, 0.901]	0.306	[0.239, 0.375]
LR-LLM	0.794	[0.756, 0.836]	0.883	[0.859, 0.908]	0.308	[0.236, 0.378]
LR-L	0.992	[0.981, 1.000]	0.995	[0.990, 1.000]	0.011	[0.000, 0.025]

TABLE VIII

Holm-Bonferroni-Corrected Paired Bootstrap Significance Tests (7 pre-specified comparisons)

Comparison	Metric	Mean Diff	95% CI	Uncorrected p	Corrected alpha	Significant?
LR-H vs. PR-Adapted	Top-1	+0.728	[+0.681, +0.769]	<0.001 +	0.0071	Yes
LR-L vs. LR-H	Top-1	+0.264	[+0.219, +0.311]	<0.001 +	0.0083	Yes
LR-CP vs. Quality only	Top-1	-0.089	[-0.136, -0.044]	<0.001 +	0.0100	Yes
LR-BS vs. LR-H	Top-1	+0.056	[+0.014, +0.100]	0.005	0.0125	Yes
LR-LLM vs. LR-H	Top-1	+0.067	[+0.008, +0.125]	0.023	0.0167	No
LR-H vs. Centrality	Top-1	+0.061	[-0.006, +0.128]	0.080	0.0250	No
LR-LLM vs. LR-BS	Top-1	+0.011	[-0.036, +0.061]	0.677	0.0500	No

+p < 0.001 represents $\leq 2/1,500$ samples; exact bootstrap floor is p < 0.00067.

E. Observability Breakdown

TABLE IX

Performance by Observability Condition (Top-1 / MRR)

Method	Full	Runtime-Sparse	Runtime-Missing-Root
LR-H	0.675 / 0.826	0.717 / 0.842	0.792 / 0.888
LR-BS	0.575 / 0.760	0.775 / 0.874	1.000 / 1.000
LR-LLM	0.642 / 0.804	0.800 / 0.881	0.942 / 0.963
LR-L	0.983 / 0.990	0.992 / 0.996	1.000 / 1.000

LR-BS is a partial-observability specialist: under full observability, blind_spot_hint = 0 for all nodes and LR-BS (0.575) substantially trails LR-H (0.675). Under runtime-missing-root, Proposition 1's source-node guarantee holds for all 120/120 incidents (Top-1 = 1.000, MRR = 1.000). LR-LLM (0.642) underperforms LR-H (0.675) under full observability — the structural anchor lineage_rank is calibrated for partial-observability conditions, and at alpha = 0.60 the LLM weight pulls scores away from proximity-correct rankings when the graph is fully observed. Under

runtime-sparse, LR-LLM (0.800) and LR-BS (0.775) both exceed LR-H (0.717), with LLM chain-of-thought compensating for dropped edges. LR-L achieves Top-1 = 1.000 under both runtime-sparse and missing-root by leveraging design_proximity and blind_spot_hint features unaffected by runtime edge dropout.

F. Per-Pipeline Breakdown

TABLE X

Per-Pipeline Top-1 Accuracy

Method	Yellow Taxi	Green Taxi	Divvy Bike	BTS Airline
LR-H	0.689	0.744	0.700	0.778
LR-BS	0.833	0.800	0.667	0.833
LR-LLM	0.900	0.722	0.722	0.833
LR-L	0.989	0.989	1.000	0.989

LR-L achieves ≥ 0.989 across all four pipelines (perfect on Divvy), reflecting strong LOPO generalisation. LR-LLM achieves its highest accuracy on Yellow Taxi (0.900) where trip-volume and zone semantics are well-represented in model priors, but is substantially lower on Green and Divvy (0.722), suggesting that station-based join-key and smaller-dataset semantics are less well-captured by the LLM. LR-BS shows wide cross-pipeline variance (0.667 Divvy to 0.833 Yellow/BTS), reflecting sensitivity to whether source-root faults dominate within each pipeline's random draws.

G. Fault-Type Analysis

LR-L achieves Top-1 = 1.000 on five of six fault types; schema_drift is the only miss (Top-1 = 0.950), reflecting that structural schema changes produce subtler row-count anomaly signals.

LR-H proximity-bias failure on null_explosion (Top-1 = 0.050): Null explosions propagate NULL values through join chains, generating anomaly signals at every downstream node. In 4-hop pipelines, proximity weighting assigns higher scores to nodes closer to v_obs — the victim nodes, not the source. LR-BS partially mitigates this (Top-1 = 0.467) via blind-spot amplification. **LR-LLM substantially resolves it (Top-1 = 0.817)** via chain-of-thought propagation reasoning that identifies the null-propagation root without proximity guidance. LR-L eliminates it entirely (Top-1 = 1.000).

Fault-type by method (Top-1):

Fault Type	LR-H	LR-BS	LR-LL M	LR-L	LR-CP
stale_source	1.000	1.000	0.967	1.000	0.883
missing_partition	0.883	0.850	0.800	1.000	0.083
duplicate_ingestion	0.933	0.800	0.833	1.000	0.100
null_explosion	0.050	0.467	0.817	1.000	0.017
bad_join_key	0.600	0.600	0.883	1.000	0.200
schema_drift	0.900	0.983	0.467	0.950	1.000

LR-LLM's largest gains over LR-H are on null_explosion (+0.767) and bad_join_key (+0.283), where chain-of-thought propagation reasoning provides structural insight beyond proximity heuristics. LR-LLM severely underperforms on schema_drift (0.467 vs. LR-H 0.900) — schema changes produce syntactic contract violations that heuristic features detect reliably but that LLM prompts framed around row-count anomalies may misinterpret. LR-CP achieves 1.000 on schema_drift (contract violations are unambiguous) but collapses on row-count faults (missing_partition 0.083, null_explosion 0.017).

H. Feature Importance

Top-5 LR-L importances (averaged across LOPO folds):

Rank	Feature	Importance
1	recent_change	0.184
2	contract_violation	0.159
3	run_anomaly	0.112
4	design_proximity	0.104
5	proximity	0.091

Under calibrated signals, recent_change (0.184) rises to the top feature — reflecting that with Type A fault signals anchored to real row-count deltas, the source node's recent-change indicator becomes more discriminative than raw anomaly magnitude. contract_violation (0.159) retains second place. run_anomaly (0.112) drops to third: calibrated signals narrow the gap between root and decoy anomaly ranges, reducing its single-feature discriminative power. design_proximity (0.104) enters the top five, consistent with its role under missing-root conditions. The shift from a run_anomaly-dominant to a multi-feature profile confirms that the calibrated benchmark requires structural and freshness features to disambiguate root from decoy.

I. Baseline Failure Mode Analysis

The seven individual baselines share correlated failure modes — particularly on null_explosion and bad_join_key where all

proximity and distance signals fail simultaneously. Runtime-distance and design-distance both score Top-1 = 0.000–0.006, confirming that graph distance alone is insufficient for pipeline RCA. Quality-only (Top-1 = 0.469) achieves modest performance by exploiting contract violation and freshness signals but cannot compensate for the proximity-bias failure that is resolved by LR-L. LR-CP (0.381) significantly underperforms Quality-only ($p < 0.001$, corrected), confirming that gradient estimation over correlated signals introduces noise rather than signal.

J. Ablation Study

TABLE XI

LR-H Component Ablation (36-incident pilot evaluation)

Variant	Top-1	MRR	Δ vs. Full
Full (16 features)	0.722	0.847	—
Without fault_prior	0.778	0.866	+0.056
Without blind_spot_hint	0.389	0.612	-0.333
Without proximity family	0.556	0.755	-0.167

Note: Ablation numbers are from the 36-incident held-out pilot evaluation; full-benchmark LR-H = 0.728. The fault_prior reversal (+0.056) warrants further investigation with a larger sample.

Removing blind_spot_hint causes the largest performance drop (-0.333 Top-1), making it the single most critical feature in LR-H's additive score. Counter-intuitively, removing fault_prior improves performance (+0.056 Top-1) on the pilot set — suggesting that the domain-knowledge prior encoded in fault_prior is miscalibrated for the calibrated signal injection ranges, where stochastic run_anomaly distributions already discriminate fault types sufficiently. Removing the proximity family causes a notable drop (-0.167). Blind-spot hint is the essential component; fault_prior should be re-examined or removed pending recalibration.

K. LR-LLM Results

LR-LLM achieves **Top-1 = 0.794**, **MRR = 0.883**, **nDCG = 0.913**, and **Avg. Assets = 0.308** across all 360 incidents, scored using Claude Sonnet 4.5 via a litellm proxy with 100% call success rate (360/360 live, 0 fallbacks). All 360 LLM calls completed successfully; no fallback-to-structural scoring was required.

Comparison with heuristics: LR-LLM achieves a directional gain over LR-H of +0.067 pp Top-1 (95% CI [+0.008, +0.125], $p = 0.023$), but this does **not survive Holm-Bonferroni correction** (corrected threshold 0.0167). The gain is most pronounced under runtime-sparse observability, where LR-LLM (Top-1 = 0.800) exceeds LR-H (0.717) by +8.3 pp — consistent with the hypothesis that LLM chain-of-thought reasoning compensates for dropped runtime edges. Under full observability, LR-LLM (0.642) trails LR-H (0.675): at alpha = 0.60 the LLM weight pulls scores away from proximity-correct rankings when the graph is fully observed.

Comparison with LR-BS: LR-LLM (0.794) versus LR-BS (0.783) yields a difference of +0.011 pp that is **not statistically significant** ($p = 0.677$; 95% CI [-0.036, +0.061]). LR-LLM and LR-BS are statistically indistinguishable. This finding reveals that LR-LLM's gain over LR-H originates primarily from its ability to detect blind-spot and propagation-path structure — behavior already captured analytically by LR-BS's multiplicative amplification. The structural anchor (lineage_rank, weight $1 - \alpha = 0.40$) is load-bearing: at $\alpha = 1.0$ (LLM-only, no structural anchor), pilot Top-1 = 0.833 — above the calibrated tuned result (0.722) on pilot but the benchmark run uses $\alpha = 0.60$.

By observability: LR-LLM achieves Top-1 = 0.642 (full), 0.800 (sparse), 0.942 (missing-root). The gap vs. LR-BS under missing-root (0.942 vs. 1.000) represents approximately 7 incidents where LLM reasoning failed to identify the design-absent source node as the root cause, likely because prompt context did not sufficiently emphasize the missing-edge signature.

By pipeline: LR-LLM ranges from 0.722 (Green, Divvy) to 0.900 (Yellow Taxi), with high cross-pipeline variance. Yellow Taxi's trip-volume and NYC zone semantics appear best-represented in the model's domain priors.

Operational cost: 360 API calls at $\sim 3s$ per call ≈ 18 minutes for the full benchmark. At inference time, LR-LLM adds approximately 2–4 seconds latency per incident diagnosis. For synchronous debugging workflows, this is acceptable; for high-throughput automated triage, LR-BS (zero LLM calls, statistically equivalent overall) is preferred.

L. Worked Example: null_explosion on BTS Airline Pipeline

To illustrate the proximity-bias failure and its resolution, we trace one incident from the BTS pipeline: fault type null_explosion, root cause raw_flights (source node), observed failure at delay_tier_metrics (mart), full observability condition.

Candidate set (6 nodes, sorted by ground-truth distance from root): raw_flights (d=1), flights_valid (d=2), airport_lookup (d=2), flights_enriched (d=3), flights_classified (d=4), delay_tier_metrics (d=5, observed failure).

Feature values (selected):

Node	run_anomaly	blind_spot	proximity	LR-H rank	LR-BS rank	LR-L rank
raw_flights (root)	0.71	0	0.50	3	3	1 [OK]
flights_valid	0.68	0	0.67	2	2	2
flights_enriched	0.65	0	0.83	1 [X]	1 [X]	3
flights_classified	0.58	0	0.50	4	4	4

LR-H and LR-BS both rank flights_enriched first because its proximity (0.83, one hop from observed failure) dominates over raw_flights (0.50, four hops). LR-L ranks raw_flights first by recognizing that run_anomaly = 0.71 at the source node, combined with design_proximity patterns learned across all fault types, correctly identifies the root despite its distance from v_obs. This example illustrates why proximity-bias is a structural failure: in full-observability null_explosion faults, the blind_spot signal is 0 for all nodes, leaving LR-BS unable to distinguish root from victim.

VII. DISCUSSION

A. Practical Recommendations

LR-L: recommended for deployments with a training corpus (Top-1 0.992, Avg. Assets 0.011 — near-first-try diagnosis across all fault types, with Divvy at perfect 1.000).

LR-LLM: recommended where LLM API access is available, especially for null_explosion (Top-1 0.817, +0.767 vs. LR-H) and bad_join_key faults (0.883), where chain-of-thought propagation reasoning substantially mitigates proximity bias. Statistically indistinguishable from LR-BS overall ($p = 0.677$). Avoid for schema_drift faults (Top-1 0.467) where contract-violation heuristics dominate. Requires ~ 2 –4s latency per incident.

LR-BS: recommended interpretable zero-LLM heuristic under root-absence conditions, where Proposition 1 guarantees Top-1 = 1.000 for source-node roots (fully confirmed: 120/120 incidents — Top-1 0.783 overall, Avg. Assets 0.306, statistically significant vs. LR-H at $p = 0.005$). Under full observability, LR-BS (0.575) trails LR-H (0.675) substantially — only deploy LR-BS when partial observability is anticipated. The guarantee does not extend to staging-node roots.

LR-H: appropriate as a lightweight zero-training baseline across all observability conditions — but practitioners should note the null_explosion proximity bias (Top-1 0.050) and consider removing fault_prior (pilot ablation shows +0.056 gain without it).

B. Limitations

Proximity bias in LR-H: structural limitation of additive scoring in multi-hop null-propagation pipelines. Unaffected by LR-L and resolved by LR-BS under runtime_missing_root.

LR-BS full-observability gap: Under full observability, LR-BS (0.575) substantially trails LR-H (0.675) because blind_spot_hint = 0 for all nodes and LR-BS reduces to its base score. LR-BS should only be deployed when partial observability (runtime-missing-root or runtime-sparse) is the expected operational condition. Under missing-root, Proposition 1's guarantee holds for all 120/120 source-node-root incidents in PipeRCA-Bench (Top-1 = 1.000); the guarantee does not extend to staging-node roots.

LR-CP gradient fragility: LR-CP (0.381) significantly underperforms Quality-only (0.469) by -0.089 pp ($p < 0.001$, Holm-Bonferroni corrected threshold 0.010), confirming that gradient estimation actively degrades plain evidence aggregation on this benchmark. Correlated evidence distributions across upstream nodes make gradient estimation unreliable; temporal evidence

windows (e.g., comparing run_anomaly across pipeline execution batches) remain future work.

30% sparse dropout: an unvalidated design parameter. Future calibration against real OpenLineage deployment event-loss rates would strengthen the observability condition design.

Single-root-cause assumption: compound failures are excluded. Foidl et al. [1] support this for the fault types studied, but multi-root failures in practice deserve future benchmark work.

Pipeline scale: all evaluated pipelines have 8 nodes; real enterprise pipelines typically have 50–500 nodes. LineageRank's computational complexity is $O(|C|)$ per incident (linear in candidate set size), but rank quality and signal discrimination under larger candidate sets are untested.

Practitioner validation: the benchmark's ground truth is constructed; no user study evaluates whether ranked outputs are operationally actionable. Expert annotation of ranked lists is a planned future step.

Signal separability: root node signals (run_anomaly in [0.48, 0.84]) and decoy signals ([0.34, 0.71]) overlap in [0.48, 0.71], creating genuine ambiguity. LR-L's near-perfect performance confirms that the Random Forest learns structure beyond naive thresholding, but performance on naturally noisier real signals may differ.

C. Generalizability

LOPO cross-validation across four structurally distinct pipeline families (including one novel dual-path DAG) confirms LR-L generalizes across topology types. Extension to streaming pipelines and graph-database lineage remains future work.

VIII. CONCLUSION

We introduced PipeRCA-Bench (360 real-data incidents, four pipeline families, six fault classes, three observability conditions) and the LineageRank framework of five ranking methods. Key findings: (1) LR-L achieves Top-1 0.992, effectively eliminating proximity bias across all fault types with near-perfect LOPO generalisation; (2) LR-BS achieves Top-1 0.783 overall (significant vs. LR-H, $p = 0.005$) and Top-1 1.000 under root-absence conditions (120/120, fully confirming Proposition 1 for source-node roots); under full observability LR-BS (0.575) trails LR-H (0.675) — it is a partial-observability specialist; (3) LR-LLM achieves Top-1 0.794 and is statistically indistinguishable from LR-BS ($p = 0.677$), confirming that LLM chain-of-thought reasoning replicates structural blind-spot detection; LR-LLM's directional gain over LR-H (+0.067 pp, $p = 0.023$) does not survive Holm-Bonferroni correction; LR-LLM's largest contribution is on null_explosion (Top-1 0.817, +0.767 vs. LR-H) via propagation-path reasoning; (4) LR-H suffers a structural proximity-bias failure on null_explosion (Top-1 0.050), mechanistically explained and fully resolved by LR-LLM and LR-L; (5) LR-CP (0.381) is significantly worse than Quality-only (0.469) by -0.089 pp ($p < 0.001$, corrected), confirming evidence gradient estimation actively degrades plain evidence aggregation without temporal windows; (6) PR-Adapted

scores 0.000 Top-1, confirming pipeline DAGs require purpose-built evidence-fusion methods; (7) removing fault_prior improves pilot LR-H performance (+0.056 Top-1), flagging miscalibrated domain priors as a target for future recalibration. All methods, data, and results are released for reproducibility.

REFERENCES

- [1] H. Foidl, M. Felderer, and R. Ramler, "Data smells in public datasets," in *Proc. Int. Conf. AI Engineering*, ACM, 2022.
- [2] OpenLineage Specification, The Linux Foundation, 2022. [Online]. Available: <https://openlineage.io>
- [3] dbt Core, dbt Labs, 2024. [Online]. Available: <https://docs.getdbt.com>
- [4] dbt Contracts, dbt Labs, 2023.
- [5] dbt Tests, dbt Labs, 2024.
- [6] dbt Freshness, dbt Labs, 2024.
- [7] Apache Airflow Dataset-aware scheduling, Apache Software Foundation, 2023.
- [8] OpenMetadata Incident Management, 2024.
- [9] OpenLineage Static Lineage, The Linux Foundation, 2023.
- [10] OpenLineage Column-Level Lineage, The Linux Foundation, 2023.
- [11] Z. Yu et al., "RCAEval: A benchmark for root cause analysis of microservice systems," *arXiv:2403.12177*, 2024.
- [12] A. Chapman, E. Curry, and H. Sherif, "A provenance model for data science pipelines," in *Proc. EDBT*, 2023.
- [13] S. Schelter, F. Biessmann, and T. Januschowski, "On challenges in machine learning model management," *IEEE Data Eng. Bull.*, vol. 41, no. 4, 2018.
- [14] M. Johns et al., "Provenance-integrated clinical ETL quality dashboards," *J. Am. Med. Inform. Assoc.*, vol. 30, no. 6, 2023.
- [15] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual modeling for ETL processes," in *Proc. EDBT*, 2002.
- [16] M. Barth, F. Naumann, and E. Müller, "Data staleness in data lakes: Empirical findings," in *Proc. BTW*, 2023.
- [17] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.
- [18] M. Li et al., "CIRCA: Causal interpretation for root cause analysis," in *Proc. ICDM*, 2022.
- [19] L. Xin et al., "CausalRCA: Causal inference-based root cause analysis for microservices," in *Proc. ICSOC*, 2023.
- [20] X. Wang et al., "PRISM: Graph-free root cause analysis for microservices," in *Proc. ICSE*, 2024.
- [21] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [22] NYC Taxi & Limousine Commission, "TLC Trip Record Data," 2024. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [23] L. Page et al., "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999-66, 1999.
- [24] G. Chen et al., "Observability gaps in microservice fault injection benchmarks," in *Proc. ISSRE*, 2025.

- [25] J. Soldani et al., "An empirical comparison of root cause analysis approaches for microservice systems," *J. Syst. Softw.*, vol. 210, 2024.
- [26] Z. Chen et al., "DiagGPT: An LLM-based chatbot for cloud incident management," in *Proc. SoCC*, 2023.
- [27] Z. Chen et al., "RCACopilot: On-call LLM agent for incident root cause analysis," in *Proc. FSE*, 2024.
- [28] Y. Shan et al., "Towards automated log-based anomaly detection and diagnosis," in *Proc. ICSE*, 2024.
- [29] Lyft Inc., "Divvy Bikes trip data," 2024. [Online]. Available: <https://divvybikes.com/system-data>
- [30] U.S. Bureau of Transportation Statistics, "Airline On-Time Performance Data," January 2024. [Online]. Available: <https://transtats.bts.gov>